

A Practical Guide to Proximal policy optimization

TOPIC -- PROXIMAL POLICY OPTIMIZATION

-- 01 --

Stability While other RL algorithms require hyperparameter tuning, PPO comparatively does not require as much (0.2 for epsilon can be used in most cases). Also, PPO does not require sophisticated optimization techniques. It can be easily practiced with standard deep learning frameworks and generalized to a broad range of tasks.

-- 02 --

Ratio function In PPO, the ratio function (r_t) calculates the probability of selecting action a in state s given the current policy network, divided by the previous probability under the old policy. In other words:

-- 03 --

$r_t(\theta) \cdot A$: this is the product of the ratio function and the advantage function introduced in TRPO, also known as the normal policy gradient objective.

-- 04 --

PPO objective function The objective function of PPO takes the expectation operator (denoted as E) which means that this function will be computed over quantities of trajectories. The expectation operator takes the minimum of two terms:

-- 05 --

for $k = 0, 1, 2, \dots$ do Collect set of trajectories $D_k = \{ \tau_i \}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment. Compute rewards-to-go $\hat{R}_t$. Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k} . Update the policy by maximizing the PPO-Clip objective: $\theta_{k+1} = \arg \max_{\theta} \frac{1}{|D_k|} \sum_{\tau \in D_k} \sum_{t=0}^T \min(\pi_{\theta}(a_t | s_t) \pi_{\theta_k}(a_t | s_t) \hat{A}_t, \pi_{\theta_k}(s_t, a_t), g(s_t, a_t))$

$\{D\}_{k+1} = \arg \min_{\phi} \frac{1}{T} \sum_{\tau \in D_k} \sum_{t=0}^T (V_{\phi}(s_t) - R_t)^2$ typically via stochastic gradient ascent with Adam. Fit value function by regression on mean-squared error: $\phi_{k+1} = \arg \min_{\phi} \frac{1}{T} \sum_{\tau \in D_k} \sum_{t=0}^T (V_{\phi}(s_t) - \hat{R}_t)^2$ typically via some gradient descent algorithm. Like all policy gradient methods, PPO is used for training an RL agent whose actions are determined by a differentiable policy function by gradient ascent. Intuitively, a policy gradient method takes small policy update steps, so the agent can reach higher and higher rewards in expectation. Policy gradient methods may be unstable: A step size that is too big may direct the policy in a suboptimal direction, thus having little possibility of recovery; a step size that is too small lowers the overall efficiency. To solve the instability, PPO implements a clip function that constrains the policy update of an agent from being too large, so that larger step sizes may be used without negatively affecting the gradient ascent process.

-- 06 --

Use the conjugate gradient algorithm to compute $\hat{x}^k \approx H^{-1} g^k$ where H^k is the Hessian of the sample average KL-divergence. Update the policy by backtracking line search with $\theta_{k+1} = \theta_k + \alpha_j \delta x^k$ where $j \in \{0, 1, 2, \dots, K\}$ is the smallest value which improves the sample loss and satisfies the sample KL-divergence constraint. Fit value function by regression on mean-squared error: $\phi_{k+1} = \arg \min_{\phi} \frac{1}{T} \sum_{\tau \in D_k} \sum_{t=0}^T (V_{\phi}(s_t) - \hat{R}_t)^2$ typically via some gradient descent algorithm.

-- 07 --

$\text{clip}(\pi_{\theta'} / \pi_{\theta}) \cdot A$: the policy ratio is first clipped to the range $[1 - \epsilon, 1 + \epsilon]$; generally, ϵ is defined to be 0.2. Then, as before, it is multiplied by the advantage. The fundamental intuition behind PPO is

the same as that of TRPO: conservatism. Clipping results in a conservative advantage estimate of the new policy. The reasoning is that if an agent makes significant changes due to high advantage estimates, its policy update will be large and unstable, and may diverge from the optimal policy with little possibility of recovery. There are two common applications of the clipping function: when an action under a new policy happens to be a good choice based on the advantage function, the clipping function limits how much credit can be given to a new policy for up-weighted good actions. On the other hand, when an action under the old policy is judged to be bad, the clipping function constrains how much the agent can accept the down-weighted bad actions of the new policy. Consequently, the clipping mechanism is designed to discourage the incentive of moving beyond the defined range by clipping both directions. The advantage of this method is that it can be optimized directly with gradient descent, as opposed to the strict KL divergence constraint of TRPO, making the implementation faster and more intuitive. After computing the clipped surrogate objective function, the agent has two probability ratios: one non-clipped and one clipped. Then, by taking the minimum of the two objectives, the final objective becomes a lower bound (a pessimistic bound) of what the agent knows is possible. In other words, the minimum method makes sure that the agent is doing the safest possible update.